



SOFTWARE DEV

MERN STACK DEVELOPMENT REPORT

Week 5 : Routing & Navigation in React

Muneeb Ur Rehman

<https://github.com/955muneeb/Glaxit-internship/tree/main/week-5/week5-react-app/week5-react-app>

Laboratory Report – Week 5, Day 5

Dynamic Routing & URL Parameters in React

Abstract

This laboratory exercise focused on implementing **dynamic routing** in a React application using the **react-router-dom** library. In the previous lab, we created fixed routes such as **Home** and **About**. In this exercise, we extended the routing system by creating dynamic URLs that display different product details based on the selected product. We also learned how to read URL parameters using the **useParams** hook and how to handle invalid URLs gracefully by displaying a **Product Not Found** message. This exercise demonstrated how real-world React applications display different content using a single route.

Objectives

The objectives of this laboratory exercise were:

- To understand dynamic routing in React.
 - To learn how URL parameters work.
 - To use the **useParams** hook to read dynamic values from the URL.
 - To create a Products page with multiple product cards.
 - To display product details based on the selected product.
 - To handle invalid product IDs without crashing the application.
-

Introduction to Dynamic Routing

In the previous lab, we worked with **fixed routes** such as:

- `/`
- `/about`

These URLs always display the same page.

However, most real-world websites contain pages where part of the URL changes depending on the selected item. For example:

```
/products/1  
/products/2  
/products/3
```

Although the URLs are different, they all use the same React component. The only thing that changes is the product ID.

This concept is called **dynamic routing**. Instead of creating a separate route for every product, React allows one route to display unlimited products by using URL parameters.

Task Performed

The task was to build a **Products** page that displays a grid of product cards.

When the user clicks on a product, the application navigates to a new URL such as:

```
/products/1
```

or

```
/products/2
```

The application then reads the product ID from the URL and displays the corresponding product details.

If the entered product ID does not exist, the application shows a **Product Not Found** message instead of crashing.

Step-by-Step Implementation

Step 1 – Creating the Product Data

A new file named **products.js** was created.

This file contains an array of product objects. Each product stores information such as:

- Product ID
- Product Name
- Price
- Description

Keeping product information inside a separate data file makes the application more organized and easier to maintain.

Step 2 – Creating the Products Page

A **Products.jsx** page was created to display all available products.

The component uses the JavaScript **map()** function to loop through the product array and create one card for each product.

Every product card contains a **Link** component that navigates to the product's detail page.

For example, clicking Product 1 opens:

```
/products/1
```

Clicking Product 2 opens:

```
/products/2
```

This allows every product to have its own unique URL.

Step 3 – Creating a Dynamic Route

A new route was added inside **App.jsx**.

Instead of writing a separate route for every product, a dynamic route was created using:

```
/products/:id
```

The `:id` part is called a **URL parameter**.

It acts as a placeholder, meaning React accepts any value in that position and stores it as **id**.

This allows one route to handle unlimited products.

Step 4 – Reading URL Parameters

Inside **ProductDetail.jsx**, the **useParams()** hook was used.

The hook reads the value stored in the `:id` section of the URL.

For example:

- `/products/1` → `id = 1`
- `/products/5` → `id = 5`

After reading the ID, the application searches the product array using the **find()** method to locate the matching product.

If a matching product is found, its name, price, and description are displayed on the screen.

Step 5 – Handling Invalid Product IDs

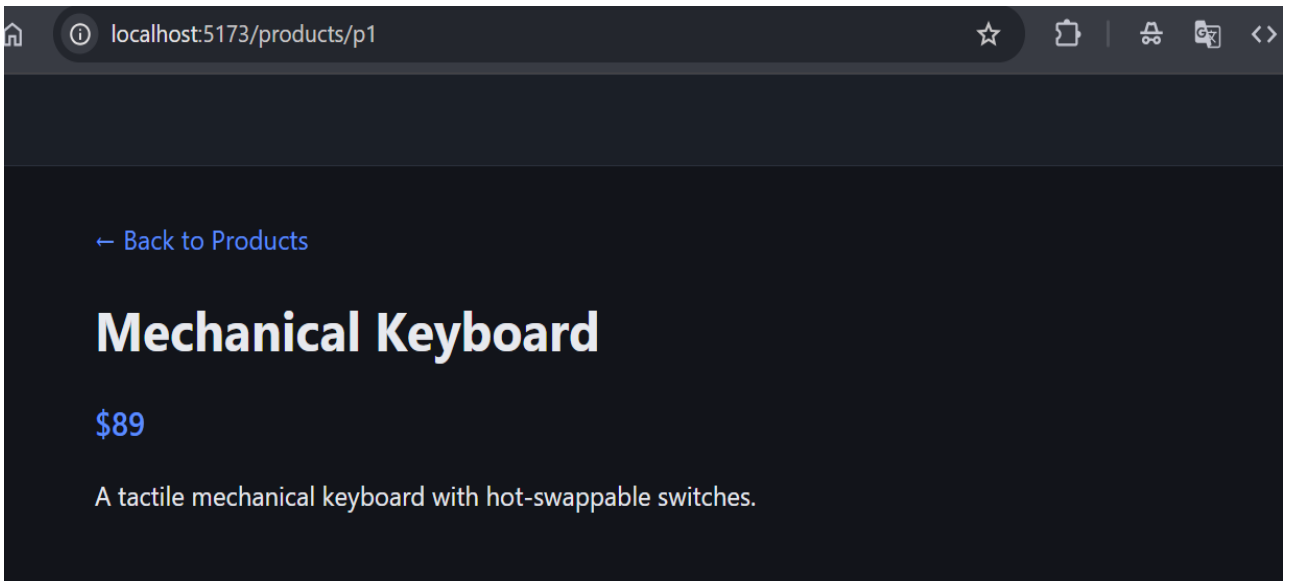
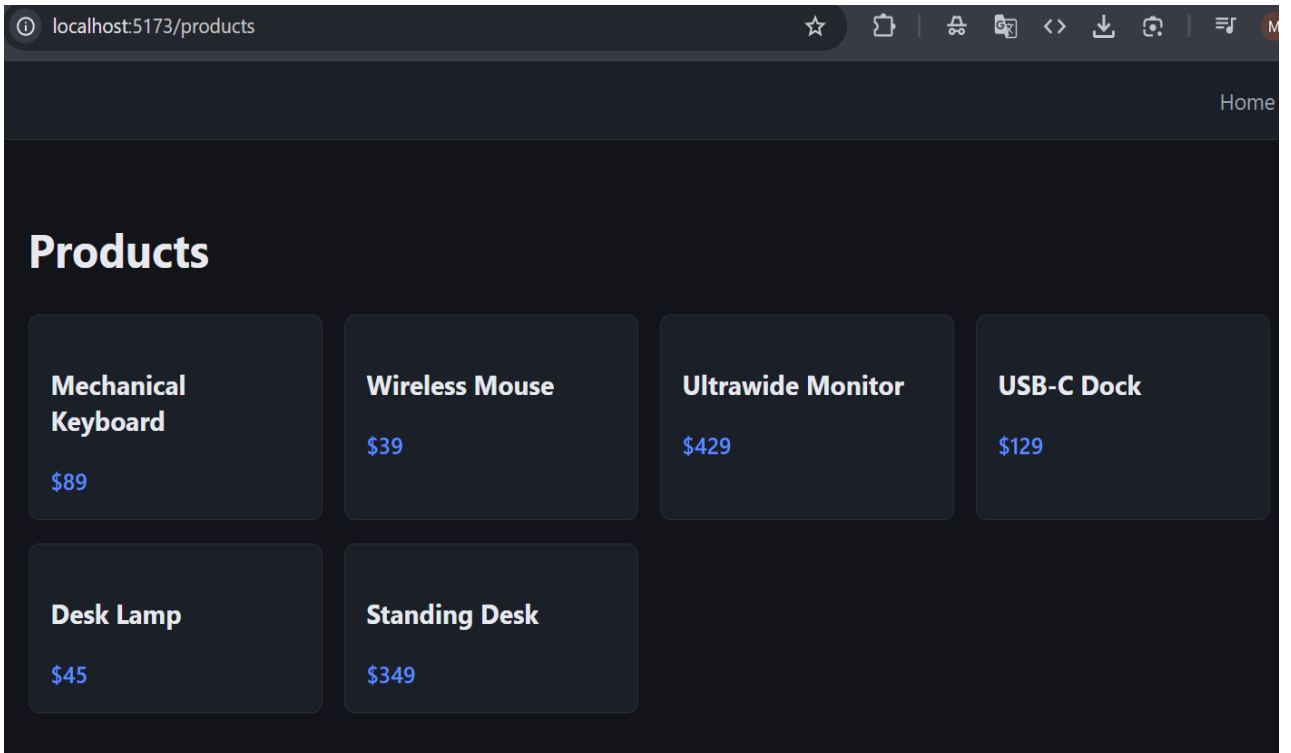
A fallback condition was added for cases where no matching product exists.

For example, if the user manually enters:

```
/products/100
```

and product 100 does not exist, the application displays a friendly **Product Not Found** message along with a link that allows the user to return to the Products page.

This prevents the application from crashing and provides a better user experience.





[← Back to Products](#)

Standing Desk

\$349

An electric height-adjustable standing desk.